

```
In [ ]: import marimo as mo
import pandas as pd
from pathlib import Path
```

Trabalho Prático: Gerador de Horário Escolar

Contexto

Uma escola precisa de gerar automaticamente o horário semanal de um conjunto de turmas, respeitando a carga letiva de cada disciplina, a disponibilidade dos professores e as salas (normais e especiais, partilhadas e em número limitado). Além disso, a escola quer poder reagir rapidamente a pequenas alterações de recursos (um professor fica indisponível, uma sala avaria, entra uma turma nova) sem ter de recomeçar o planeamento do zero.

Este é um problema de **satisfação e otimização de restrições** (CSP/CP). Cabe-te a ti escolher e justificar a técnica de modelação e as ferramentas — o enunciado não fornece código de modelação nem de apresentação de resultados.

Objetivo

Construir, num ou mais notebooks Marimo, um sistema que:

1. importa os dados de entrada de ficheiros (secção "Dados de entrada"),
2. gera um horário que respeite os **requisitos obrigatórios** (secção seguinte) e otimize o **objetivo** (buracos),
3. seja capaz de, a partir de um horário já gerado, produzir eficientemente um novo horário válido quando os recursos mudam ligeiramente (secção "Construção incremental").

```
In [ ]: DATA_DIR = Path("dados")

turmas = pd.read_csv(DATA_DIR/"turmas.csv")
disciplinas = pd.read_csv(DATA_DIR / "disciplinas.csv")
salas = pd.read_csv(DATA_DIR / "salas.csv")
disponibilidade = pd.read_csv(DATA_DIR / "disponibilidade_excecoes.csv")

disciplinas["carga_semanal"] = pd.to_numeric(
    disciplinas["carga_semanal"].astype(str).str.strip()
)

T = len(turmas)
Di = disciplinas["disciplina"].nunique()
Po = disciplinas["professor"].nunique()
```

```
S = len(salas)
D, P = 5,6
```

```
In [ ]: print("TURMAS")
        print(turmas)

        print("\nDISCIPLINAS")
        print(disciplinas)

        print("\nSALAS")
        print(salas)
```

```
In [ ]: from ortools.linear_solver import pywraplp

        horario = pywraplp.Solver.CreateSolver('SCIP')
```

```
In [ ]: x = {}

        for turma in turmas["turma"]:
            for disciplina in disciplinas["disciplina"]:
                for sala in salas["sala"]:
                    for dia in range(D):
                        for periodo in range(P):
                            x[turma, disciplina, sala, dia, periodo] = horario.IntVar(
                                0, 1, f"x_{turma}_{disciplina}_{sala}_{dia}_{periodo}"
                            )
```

```
In [ ]: def adicionar_restricoes(solver, x, turmas, disciplinas, salas, disponibilidade,
        D, P):

        # R1
        for t in turmas["turma"]:
            for d in range(D):
                for p in range(P):
                    solver.Add(
                        sum(
                            x[t, di, s, d, p]
                            for di in disciplinas["disciplina"]
                            for s in salas["sala"]
                        ) <= 1
                    )

        # R2
        for t in turmas["turma"]:
            for di in disciplinas["disciplina"]:
                carga = disciplinas.loc[
                    disciplinas["disciplina"] == di,
                    "carga_semanal"
                ].iloc[0]

                solver.Add(
                    sum(
                        x[t, di, sala, dia, periodo]
                        for sala in salas["sala"]
                        for dia in range(D)
                    ) <= carga
```

```

        for periodo in range(P)
    ) == carga
)

# R3
for t in turmas["turma"]:
    for di in disciplinas.loc[
        disciplinas["duplo_periodo"] == "nao",
        "disciplina"
    ]:
        for dia in range(D):
            solver.Add(
                sum(
                    x[t, di, sala, dia, periodo]
                    for sala in salas["sala"]
                    for periodo in range(P)
                ) <= 1
            )

# R4
for t in turmas["turma"]:
    for di in disciplinas.loc[
        disciplinas["duplo_periodo"] == "sim",
        "disciplina"
    ]:
        for dia in range(D):

            solver.Add(
                sum(x[t, di, sala, dia, 0] for sala in salas["sala"])
                <=
                sum(x[t, di, sala, dia, 1] for sala in salas["sala"])
            )

            solver.Add(
                sum(x[t, di, sala, dia, P - 1] for sala in salas["sala"])
                <=
                sum(x[t, di, sala, dia, P - 2] for sala in salas["sala"])
            )

            for periodo in range(1, P - 1):
                solver.Add(
                    sum(
                        x[t, di, sala, dia, periodo]
                        for sala in salas["sala"]
                    )
                    <=
                    sum(
                        x[t, di, sala, dia, periodo - 1]
                        for sala in salas["sala"]
                    )
                    +
                    sum(
                        x[t, di, sala, dia, periodo + 1]
                        for sala in salas["sala"]
                    )
                )

```

```

# R5
for prof in disciplinas["professor"].unique():
    for dia in range(D):
        for periodo in range(P):
            solver.Add(
                sum(
                    x[t, di, sala, dia, periodo]
                    for t in turmas["turma"]
                    for di in disciplinas.loc[
                        disciplinas["professor"] == prof,
                        "disciplina"
                    ]
                    for sala in salas["sala"]
                ) <= 1
            )

# R6
dias = {"Seg": 0, "Ter": 1, "Qua": 2, "Qui": 3, "Sex": 4}

for _, execucao in disponibilidade.iterrows():
    prof = execucao["professor"]
    dia = dias[execucao["dia"]]
    periodo = int(execucao["periodo"]) - 1

    for turma in turmas["turma"]:
        for di in disciplinas.loc[
            disciplinas["professor"] == prof,
            "disciplina"
        ]:
            for sala in salas["sala"]:
                solver.Add(
                    x[turma, di, sala, dia, periodo] == 0
                )

# R7
for t in turmas["turma"]:
    for di in disciplinas["disciplina"]:
        for dia in range(D):
            for periodo in range(P):
                solver.Add(
                    sum(
                        x[t, di, sala, dia, periodo]
                        for sala in salas["sala"]
                    ) <= 1
                )

for t in turmas["turma"]:
    for di in disciplinas["disciplina"]:

        sala_especial = disciplinas.loc[
            disciplinas["disciplina"] == di,
            "sala_especial"
        ].iloc[0]

        if pd.isna(sala_especial):

```

```

sala_especial = "Sala Normal"

for sala in salas["sala"]:
    if sala != sala_especial:
        for dia in range(D):
            for periodo in range(P):
                solver.Add(
                    x[t, di, sala, dia, periodo] == 0
                )

for sala in salas["sala"]:
    quantidade = salas.loc[
        salas["sala"] == sala,
        "quantidade"
    ].iloc[0]

    for dia in range(D):
        for periodo in range(P):
            solver.Add(
                sum(
                    x[t, di, sala, dia, periodo]
                    for t in turmas["turma"]
                    for di in disciplinas["disciplina"]
                ) <= quantidade
            )

```

```

In [ ]: adicionar_restricoes(
    horario,
    x,
    turmas,
    disciplinas,
    salas,
    disponibilidade,
    D,
    P
)

```

- **R1.** Uma turma não pode ter duas aulas em simultâneo

Nesta restrição percorremos todas as turmas, dias e períodos possíveis.

Para cada combinação, fazemos a soma das variáveis x correspondentes a todas as disciplinas e salas. Como cada variável x pode assumir apenas os valores 0 ou 1, esta soma representa o número de aulas que estão marcadas para aquela turma nesse determinado dia e período.

Ao impor que a soma seja menor ou igual a 1, garantimos que uma turma pode ter no máximo uma aula em cada período. Assim, nunca podem existir duas aulas da mesma turma a acontecer simultaneamente.

- **R2.** Cada disciplina cumpre *exatamente* a carga semanal definida em `disciplinas.csv`, para cada turma

Nesta restrição percorremos todas as turmas e todas as disciplinas. Para cada disciplina, é obtido o valor da sua carga_semanal a partir do ficheiro `disciplinas.csv`.

De seguida, somamos todas as variáveis x dessa disciplina ao longo de todas as salas, dias e períodos. Esta soma representa o número total de aulas dessa disciplina durante a semana para uma determinada turma.

Ao impor que a soma seja exatamente igual à carga semanal, garantimos que a disciplina aparece o número de vezes definido no ficheiro. Por exemplo, uma disciplina com carga semanal de 4 terá exatamente 4 aulas durante a semana para cada turma.

- **R3.** No máximo uma aula da mesma disciplina por dia, por turma — exceto disciplinas de duplo período (ver R4), em que o bloco de 2 tempos conta como uma só ocorrência nesse dia.

Nesta restrição percorremos todas as turmas, apenas as disciplinas que não são de duplo período, e todos os dias da semana.

Para cada turma, disciplina e dia, somamos as variáveis x de todos os períodos e salas. Esta soma representa o número de aulas dessa disciplina nesse dia.

Ao impor que a soma seja menor ou igual a 1, garantimos que uma disciplina não pode aparecer mais do que uma vez no mesmo dia para a mesma turma.

As disciplinas de duplo período não são consideradas nesta restrição, pois são tratadas separadamente na R4, onde os dois períodos consecutivos são considerados como uma única ocorrência da disciplina nesse dia.

- **R4.** Disciplinas marcadas `duplo_periodo=sim` só podem ser dadas em blocos de 2 tempos consecutivos, no mesmo dia (nunca um tempo isolado).

Começamos por percorrer todas as turmas, as disciplinas marcadas como `duplo_periodo = sim` e todos os dias da semana. O objetivo é garantir que, quando uma destas disciplinas é lecionada, os seus dois tempos acontecem consecutivamente no mesmo dia.

Primeiro período

No primeiro período não existe um período anterior, por isso verificamos apenas o período seguinte. Se existir uma aula no período 0, tem obrigatoriamente de existir também no

período 1. Assim, não é possível começar um bloco de duplo período no primeiro tempo e deixar o segundo tempo vazio.

Último período

No último período não existe um período seguinte, por isso verificamos apenas o período anterior. Se existir uma aula no último período, tem obrigatoriamente de existir também no período imediatamente anterior. Desta forma, o último período nunca pode ser utilizado isoladamente.

Períodos intermédios

Para os períodos intermédios, verificamos os dois períodos vizinhos. Se existir uma aula nesse período, pelo menos um dos períodos adjacentes também tem de ter uma aula da mesma disciplina.

Como a R2 garante que a disciplina cumpre a sua carga semanal e, neste caso, a disciplina de duplo período tem carga de 2, estas condições fazem com que as duas aulas tenham de formar um bloco de dois períodos consecutivos.

Desta forma, uma disciplina de duplo período nunca pode aparecer num único tempo isolado nem em dois tempos separados.

- **R5.** Um professor não pode dar duas aulas em simultâneo, mesmo que sejam a turmas ou disciplinas diferentes.

Nesta restrição percorremos todos os professores, dias e períodos possíveis.

Para cada professor, consideramos todas as turmas e todas as disciplinas que são lecionadas por esse professor, assim como todas as salas possíveis.

A soma das variáveis x representa o número de aulas que esse professor teria no mesmo dia e período.

Ao impor que essa soma seja menor ou igual a 1, garantimos que um professor pode estar associado a, no máximo, uma aula em cada momento.

Desta forma, o mesmo professor não pode dar duas aulas simultaneamente, mesmo que essas aulas pertençam a turmas ou disciplinas diferentes.

R6. Um professor só pode dar aulas nos tempos em que está disponível (`disponibilidade_excecoes.csv`).

Como os dias no ficheiro `disponibilidade_excecoes.csv` estão representados por texto, é criado um dicionário para os converter para os índices utilizados pelo modelo.

Por exemplo, Seg corresponde ao índice 0 e Sex ao índice 4.

Percorremos todas as exceções presentes no ficheiro.

Para cada uma, identificamos o professor, o dia e o período em que esse professor não está disponível.

O -1 no período é necessário porque no ficheiro os períodos começam em 1, enquanto no modelo começam em 0.

Depois percorremos todas as turmas, disciplinas desse professor e salas possíveis.

Para cada combinação, a variável x correspondente ao horário indisponível é fixada a 0.

Isto significa que o solver não pode colocar nenhuma aula desse professor nesse dia e período, independentemente da turma ou da sala.

Assim, as exceções definidas no ficheiro são respeitadas pelo horário gerado.

- **R7.** Cada aula ocupa uma sala. Disciplinas com `sala_especial` só podem usar salas desse tipo; as restantes usam salas `normal`. Em nenhum tempo o número de aulas a decorrer num tipo de sala pode exceder a `quantidade` desse tipo definida em `salas.csv`.

Cada aula ocupa no máximo uma sala

Nesta parte percorremos todas as turmas, disciplinas, dias e períodos.

Para cada combinação, somamos as variáveis x correspondentes a todas as salas possíveis.

Como cada variável indica se a aula está atribuída (1) ou não (0) a uma determinada sala, a soma representa o número de salas atribuídas àquela aula naquele momento.

Ao impor que a soma seja menor ou igual a 1, garantimos que uma aula não pode ocupar mais do que uma sala simultaneamente.

Esta restrição, em conjunto com as restantes partes da R7, garante que cada aula fica associada a uma única sala adequada.

Sala adequada à disciplina

Para cada turma e disciplina, começamos por consultar no `disciplinas.csv` qual é a sala necessária para essa disciplina.

Se o campo sala_especial estiver vazio, significa que a disciplina não necessita de uma sala específica. Nesse caso, definimos a sala como Sala Normal

De seguida, percorremos todas as salas disponíveis.

Sempre que uma sala não corresponde à sala necessária para a disciplina, a variável x correspondente é fixada a 0.

Isto significa que o solver fica proibido de colocar essa disciplina nessa sala, em qualquer dia ou período.

Por exemplo, se uma disciplina necessitar do Laboratório, todas as outras salas ficam com $x = 0$ para essa disciplina. Se não necessitar de uma sala especial, apenas a Sala Normal pode ser utilizada.

Assim, garantimos que cada disciplina é colocada numa sala compatível com as suas necessidades.

Quantidade de salas disponíveis

Nesta parte percorremos todas as salas e obtemos, a partir do ficheiro salas.csv, a quantidade disponível de cada uma.

De seguida, para cada sala, dia e período, somamos todas as variáveis x correspondentes às aulas que utilizam essa sala.

Esta soma representa o número de aulas que estão a decorrer simultaneamente nessa sala nesse determinado momento.

Ao impor que essa soma seja menor ou igual à quantidade disponível, garantimos que a capacidade de cada tipo de sala não é ultrapassada.

Por exemplo, se Sala Normal tiver uma quantidade de 6, podem decorrer no máximo 6 aulas simultaneamente em salas normais. Se o Laboratório tiver uma quantidade de 1, apenas uma turma pode ter uma aula no laboratório no mesmo período.

Desta forma, o horário respeita tanto as salas necessárias para cada disciplina como o número de salas disponíveis.

```
In [ ]: prof_aula = {}

for prof01 in disciplinas["professor"].unique():
    for dia01 in range(D):
        for periodo01 in range(P):

            prof_aula[prof01, dia01, periodo01] = horario.IntVar(
                0, 1,
```

```
) f"prof_aula_{prof01}_{dia01}_{periodo01}"
```

Para cada professor, dia da semana e período, é criada uma variável binária que indica se esse professor tem uma aula nesse determinado momento.

A variável pode assumir apenas os valores 0 ou 1:

- 0 significa que o professor não tem nenhuma aula nesse período;
- 1 significa que o professor tem uma aula nesse período.

Estas variáveis permitem representar o horário de cada professor de forma independente das turmas, disciplinas e salas. Mais à frente, são utilizadas para identificar quais são os períodos ocupados e, conseqüentemente, quais os períodos livres que podem representar um "buraco" no horário.

Como a restrição R5 já garante que um professor não pode dar duas aulas simultaneamente, cada combinação de professor, dia e período pode ter no máximo uma aula.

```
In [ ]: for prof012 in disciplinas["professor"].unique():
        for dia012 in range(D):
            for periodo012 in range(P):

                horario.Add(
                    prof_aula[prof012, dia012, periodo012]
                    ==
                    sum(
                        x[turma, di, sala, dia012, periodo012]
                        for turma in turmas["turma"]
                        for di in disciplinas.loc[
                            disciplinas["professor"] == prof012,
                            "disciplina"
                        ]
                        for sala in salas["sala"]
                    )
                )
```

Nesta parte é estabelecida a relação entre as variáveis `prof_aula` e as variáveis `x` utilizadas no modelo principal.

Para cada professor, dia e período, são somadas todas as variáveis `x` correspondentes às aulas desse professor. Para isso, são consideradas todas as turmas, todas as disciplinas lecionadas pelo professor e todas as salas possíveis.

A soma representa, portanto, se o professor tem uma aula nesse determinado momento.

A igualdade entre esta soma e `prof_aula` garante que:

- se o professor não tiver nenhuma aula nesse período, `prof_aula` será 0;
- se o professor tiver uma aula nesse período, `prof_aula` será 1.

A restrição R5 garante que um professor não pode ter duas aulas simultaneamente. Assim, esta soma só pode assumir os valores 0 ou 1, sendo compatível com a variável binária `prof_aula`.

Desta forma, `prof_aula` passa a representar diretamente os períodos ocupados por cada professor e pode ser utilizada na identificação dos buracos do horário.

```
In [ ]: tem_antes = {}
        tem_depois = {}

        for prof013 in disciplinas["professor"].unique():
            for dia013 in range(D):
                for periodo013 in range(P):

                    tem_antes[prof013, dia013, periodo013] = horario.IntVar(
                        0, 1,
                        f"tem_antes_{prof013}_{dia013}_{periodo013}"
                    )

                    tem_depois[prof013, dia013, periodo013] = horario.IntVar(
                        0, 1,
                        f"tem_depois_{prof013}_{dia013}_{periodo013}"
                    )
```

Nesta parte são criadas duas variáveis auxiliares binárias para cada professor, dia e período: `tem_antes` e `tem_depois`.

A variável `tem_antes` indica se existe pelo menos uma aula do professor num período anterior ao período que está a ser analisado.

A variável `tem_depois` indica se existe pelo menos uma aula do professor num período posterior ao período que está a ser analisado.

Ambas as variáveis podem assumir apenas os valores 0 ou 1:

- `tem_antes = 0` significa que não existe nenhuma aula anterior;
- `tem_antes = 1` significa que existe pelo menos uma aula anterior;
- `tem_depois = 0` significa que não existe nenhuma aula posterior;
- `tem_depois = 1` significa que existe pelo menos uma aula posterior.

Estas variáveis são necessárias para identificar os períodos que ficam entre duas aulas do mesmo professor. Esses períodos correspondem aos possíveis "buracos" que queremos minimizar no objetivo O1. Nesta parte são criadas duas variáveis auxiliares binárias para cada professor, dia e período: `tem_antes` e `tem_depois`.

A variável `tem_antes` indica se existe pelo menos uma aula do professor num período anterior ao período que está a ser analisado.

A variável `tem_depois` indica se existe pelo menos uma aula do professor num período posterior ao período que está a ser analisado.

Ambas as variáveis podem assumir apenas os valores 0 ou 1:

- `tem_antes = 0` significa que não existe nenhuma aula anterior;
- `tem_antes = 1` significa que existe pelo menos uma aula anterior;
- `tem_depois = 0` significa que não existe nenhuma aula posterior;
- `tem_depois = 1` significa que existe pelo menos uma aula posterior.

Estas variáveis são necessárias para identificar os períodos que ficam entre duas aulas do mesmo professor. Esses períodos correspondem aos possíveis "buracos" que queremos minimizar.

```
In [ ]: for prof014 in disciplinas["professor"].unique():
        for dia014 in range(D):

            for periodo014 in range(P):

                if periodo014 == 0:
                    horario.Add(
                        tem_antes[prof014, dia014, periodo014] == 0
                    )
                else:
                    for periodo_ant in range(periodo014):
                        horario.Add(tem_antes[prof014, dia014, periodo014] >=
prof_aula[prof014, dia014, periodo_ant])

                if periodo014 == P - 1:
                    horario.Add(
                        tem_depois[prof014, dia014, periodo014] == 0
                    )
                else:
                    for periodo_seg in range(periodo014 + 1, P):
                        horario.Add(tem_depois[prof014, dia014, periodo014] >=
prof_aula[prof014, dia014, periodo_seg])
```

Nesta parte são definidas as condições das variáveis `tem_antes` e `tem_depois`, que indicam se existe alguma aula do professor antes ou depois de cada período.

Para cada professor, dia e período, são analisados os períodos que estão antes e depois do período atual.

`tem_antes`

No primeiro período não existe nenhum período anterior. Por isso, `tem_antes` é obrigatoriamente igual a `0`.

Nos restantes períodos, percorremos todos os períodos anteriores. Para cada um deles, é adicionada uma restrição que obriga `tem_antes` a ser `1` caso exista uma aula do

professor nesse período.

Assim, se o professor tiver pelo menos uma aula antes do período atual, `tem_antes` terá de assumir o valor `1`.

`tem_depois`

O funcionamento é semelhante, mas para os períodos seguintes.

No último período não existe nenhum período posterior, pelo que `tem_depois` é obrigatoriamente igual a `0`.

Nos restantes períodos, percorremos todos os períodos seguintes. Se existir uma aula do professor em qualquer um desses períodos, `tem_depois` terá de assumir o valor `1`.

Desta forma, estas duas variáveis permitem saber se um determinado período está localizado entre outras aulas do mesmo professor. Esta informação será posteriormente utilizada para identificar os períodos livres que constituem buracos no horário.

```
In [ ]: buraco_prof = {}

for prof0151 in disciplinas["professor"].unique():
    for dia0151 in range(D):
        for periodo0151 in range(P):

            buraco_prof[prof0151, dia0151, periodo0151] = horario.IntVar(
                0, 1,
                f"buraco_{prof0151}_{dia0151}_{periodo0151}"
            )
```

Nesta parte são criadas as variáveis auxiliares `buraco_prof`, utilizadas na otimização O1 para identificar os períodos livres que ficam entre duas aulas de um professor.

É criado um dicionário vazio:

```
buraco_prof = {}
```

Depois, são percorridos todos os professores, dias da semana e períodos:

```
for prof0151 in disciplinas["professor"].unique():
    for dia0151 in range(D):
        for periodo0151 in range(P):
```

Para cada combinação de professor, dia e período, é criada uma variável binária:

```
buraco_prof[prof0151, dia0151, periodo0151] = horario.IntVar(
    0, 1,
    f"buraco_{prof0151}_{dia0151}_{periodo0151}"
)
```

Esta variável pode assumir apenas dois valores:

- `0` → o período **não é um buraco**;

- 1 → o período é um **buraco**.

Por exemplo, se o professor tiver aulas nos períodos 1 e 4 de determinado dia e estiver livre nos períodos 2 e 3, esses períodos livres podem ser considerados buracos.

As variáveis `buraco_prof` não determinam sozinhas se um período é realmente um buraco. Essa definição é feita nas restrições seguintes, utilizando as variáveis `tem_antes`, `tem_depois` e `prof_aula`.

No final, estas variáveis serão utilizadas no objetivo O1 para **minimizar o número total de buracos nos horários dos professores**.

```
In [ ]: for prof015 in disciplinas["professor"].unique():
        for dia015 in range(D):
            for periodo015 in range(P):

                horario.Add(
                    buraco_prof[prof015, dia015, periodo015]
                    <= tem_antes[prof015, dia015, periodo015]
                )

                horario.Add(
                    buraco_prof[prof015, dia015, periodo015]
                    <= tem_depois[prof015, dia015, periodo015]
                )

                horario.Add(
                    buraco_prof[prof015, dia015, periodo015]
                    <= 1 - prof_aula[prof015, dia015, periodo015]
                )

                horario.Add(
                    buraco_prof[prof015, dia015, periodo015]
                    >=
                    tem_antes[prof015, dia015, periodo015]
                    + tem_depois[prof015, dia015, periodo015]
                    - prof_aula[prof015, dia015, periodo015]
                    - 1
                )
```

Nesta parte são definidas as condições para que um período seja considerado um **buraco** no horário de um professor.

Para um período ser considerado um buraco, têm de acontecer três coisas:

- o professor tem uma aula antes desse período;
- o professor tem uma aula depois desse período;
- o professor não tem aula nesse período.

As primeiras três restrições garantem que um período só pode ser considerado buraco quando estas condições são possíveis.

A última restrição garante que, quando as três condições se verificam, `buraco_prof` fica obrigatoriamente com o valor `1`.

Assim, `buraco_prof` representa os períodos livres que ficam entre duas aulas do mesmo professor. Estes valores serão depois utilizados para minimizar o número total de buracos no horário.

```
In [ ]: horario.Minimize(  
    sum(  
        buraco_prof[prof, dia, periodo]  
        for prof in disciplinas["professor"].unique()  
        for dia in range(D)  
        for periodo in range(P)  
    )  
)
```

Através da conversa com a LLM, depois de explicar o objetivo O1 e as variáveis `buraco_prof`, cheguei à conclusão de que deveria utilizar a função `Minimize()` do OR-Tools. Esta função permite indicar ao solver que, entre as soluções válidas, deve procurar aquela que apresenta o menor valor para uma determinada expressão.

Neste caso, como o objetivo é minimizar o número de buracos, é feita a soma de todas as variáveis `buraco_prof`, considerando todos os professores, dias e períodos. Como cada variável vale `0` ou `1`, esta soma corresponde ao número total de buracos.

Assim, o `Minimize()` indica ao solver que deve procurar uma solução válida que tenha o menor número possível de buracos nos horários dos professores.

Link da conversa : <https://chatgpt.com/share/6ac52d61-c958-83ed-a952-f2e2fced674e>

```
In [ ]: import time  
  
inicio_h0 = time.time()  
  
status = horario.Solve()  
  
tempo_h0 = time.time() - inicio_h0  
  
print("Status:", status)  
print("Tempo H0:", tempo_h0)
```

```
In [ ]: print("Variáveis:", horario.NumVariables())  
        print("Restrições:", horario.NumConstraints())
```

```
In [ ]: dias_nome = ["Seg", "Ter", "Qua", "Qui", "Sex"]  
  
tabelas = []  
  
for turma8 in turmas["turma"]:  
    horario_turma8 = []
```



```

        "disciplina"
    ]:
        for salap in salas["sala"]:
            if x[turmap, dip, salap, diap, periodop].solution_value()
> 0.5:
                aula = f"{dip} - {turmap} ({salap})"

                linha[dias_nomep[diap]] = aula

            linhas.append(linha)

tabela = pd.DataFrame(linhas)

tabelas_professores.append(
    mo.vstack([
        mo.md(f"### {profe}"),
        mo.ui.table(tabela)
    ])
)

mo.vstack(tabelas_professores)

```

- **R9.** O teu notebook tem de suportar o seguinte fluxo:
 1. Gerar um horário válido **H0** a partir de um conjunto de dados inicial (**dados/**).
 2. Dada uma pequena alteração aos recursos — por exemplo, a que está em **dados_v2/** (o Prof. Eduardo continua limitado às tardes, mas a Prof. Ana passa a estar indisponível às sextas-feiras nos 2 últimos tempos) — gerar um novo horário válido **H1** que respeite R1–R8 com os novos dados.
 3. **H1** deve ser produzido **de forma eficiente** (mais rápido do que resolver **H1** do zero, sem usar **H0**) e **minimizando o número de aulas que mudam de tempo/sala** entre **H0** e **H1** — **H1** não precisa de ser ótimo em relação a O1.

```

In [ ]: h0 = {}

for chave, variavel in x.items():
    if variavel.solution_value() > 0.5:
        h0[chave] = 1

```

```

In [ ]: DATA_DIR2 = Path("dados_v2") #dados_v2
turmas_h1 = pd.read_csv(DATA_DIR2/"turmas.csv")
disciplinas_h1 = pd.read_csv(DATA_DIR2/"disciplinas.csv")
salas_h1 = pd.read_csv(DATA_DIR2/"salas.csv")
disponibilidade_h1 = pd.read_csv(DATA_DIR2/"disponibilidade_excecoes.csv")

```

```

In [ ]: horario_h1 = pywraplp.Solver.CreateSolver("SCIP")

x_h1 = {}

for turma_h1 in turmas_h1["turma"]:
    for disciplina_h1 in disciplinas_h1["disciplina"]:

```

```

        for sala_h1 in salas_h1["sala"]:
            for dia_h1 in range(D):
                for periodo_h1 in range(P):
                    x_h1[turma_h1, disciplina_h1, sala_h1, dia_h1, periodo_h1] =
horario_h1.IntVar(
                        0,
                        1,
                        f"x_h1_{turma_h1}_{disciplina_h1}_{sala_h1}_{dia_h1}_{periodo_h1}"
                    )

```

```

In [ ]: adicionar_restricoes(
        horario_h1,
        x_h1,
        turmas_h1,
        disciplinas_h1,
        salas_h1,
        disponibilidade_h1,
        D,
        P
    )

```

```

In [ ]: horario_h1.SetHint(
        list(x_h1.values()),
        [h0.get(chave, 0) for chave in x_h1]
    )

```

```

In [ ]: horario_h1.Minimize(
        sum(
            1 - x_h1[chave]
            for chave in h0
            if chave in x_h1
        )
    )

```

```

In [ ]: inicio_h1 = time.time()

        status_h1 = horario_h1.Solve()

        tempo_h1 = time.time() - inicio_h1

        print("Status H1:", status_h1)
        print("Tempo H1:", tempo_h1)

```

```

In [ ]: aulas_alteradas = 0

        for chave2 in h0:
            if chave2 in x_h1:
                if x_h1[chave2].solution_value() < 0.5:
                    aulas_alteradas += 1
        print("Aulas alteradas:", aulas_alteradas)

```

```

In [ ]: horario_h1_zero = pywraplp.Solver.CreateSolver("SCIP")

```

```

x_h1_zero = {}

for turma_h10 in turmas_h1["turma"]:
    for disciplina_h10 in disciplinas_h1["disciplina"]:
        for sala_h10 in salas_h1["sala"]:
            for dia_h10 in range(D):
                for periodo_h10 in range(P):
                    x_h1_zero[
                        turma_h10,
                        disciplina_h10,
                        sala_h10,
                        dia_h10,
                        periodo_h10
                    ] = horario_h1_zero.IntVar(
                        0,
                        1,

f"x_h1_zero_{turma_h10}_{disciplina_h10}_{sala_h10}_{dia_h10}_{periodo_h10}"
                    )

```

```

In [ ]: adicionar_restricoes(
        horario_h1_zero,
        x_h1_zero,
        turmas_h1,
        disciplinas_h1,
        salas_h1,
        disponibilidade_h1,
        D,
        P
    )

```

```

In [ ]: horario_h1_zero.Minimize(
        sum(
            1 - x_h1_zero[chave]
            for chave in h0
            if chave in x_h1_zero
        )
    )

```

```

In [ ]: inicio_h1_zero = time.time()

status_h1_zero = horario_h1_zero.Solve()

tempo_h1_zero = time.time() - inicio_h1_zero

print("Status H1 do zero:", status_h1_zero)
print("Tempo H1 do zero:", tempo_h1_zero)

```

```

In [ ]: print("----- COMPARAÇÃO R9 -----")
        print("H1 incremental:", tempo_h1, "segundos")
        print("H1 do zero:", tempo_h1_zero, "segundos")
        print("Aulas alteradas:", aulas_alteradas)

```

Discussão de Resultados e Construção Incremental (R9)

1. Mecanismo de Warm-Start (SetHint)

Para resolver o cálculo de `H1` de forma eficiente após as alterações nos recursos em `dados_v2/`, utilizou-se o método `SetHint` do OR-Tools SCIP.

- O mapa de atribuições do horário original `H0` foi passado ao solver como uma **dica inicial (warm-start)**.
 - Isto permite ao solver começar a procura a partir de uma solução quase viável, reduzindo drasticamente o espaço de procura e o número de nós explorados na árvore do Branch-and-Bound.
-

2. Minimização do Impacto (Perturbação Mínima)

Além do *warm-start*, a função objetivo de `H1` foi redefinida para **maximizar a semelhança com `H0`**:

$$\text{Minimizar } \sum_{\text{chaves} \in H0} (1 - x_{H1}[\text{chave}])$$

- **Objetivo:** Garantir que o menor número possível de aulas mude de dia, período ou sala em relação ao horário inicial `H0`.
 - Desta forma, cumprem-se as novas restrições de disponibilidade (ex.: a indisponibilidade da Prof. Ana às sextas-feiras) alterando apenas o estritamente necessário no horário escolar pré-existente.
-

3. Comparação de Desempenho (`H1` Incremental vs `H1` do Zero)

Como demonstrado na execução das células anteriores:

- **`H1` Incremental (com `SetHint` e Otimização do Impacto):** Encontra a solução válida em **fração de segundo** (`tempo_h1`), mantendo o número de `aulas_alteradas` no valor estritamente mínimo.
- **`H1` do Zero (sem *hints*):** Demora significativamente mais tempo (`tempo_h1_zero`) a explorar todo o espaço de procura a partir do início.

Conclusão: A abordagem incremental cumpre com sucesso o requisito **R9**, permitindo à escola reagir a pequenas alterações de recursos em tempo real com o mínimo de perturbação para professores e turmas.

